

Friedrich-Alexander-Universität Erlangen-Nürnberg

**Lehrstuhl für Multimediakommunikation und
Signalverarbeitung**

Team project

**Development of a Structure-from-Motion
Pipeline**

Mingyang Li , Zihua Lin , Jing Wang , Zehong Shi ,
Jiahui Shen

02.2025

Supervisor: PD Dr.-Ing. J. Seiler



Lehrstuhl für Multimediakommunikation
und Signalverarbeitung
Prof. Dr.-Ing. André Kaup



Friedrich-Alexander-Universität
Erlangen-Nürnberg

Team Project

for

Jiahui Shen, Mingyang Li, Zihua Lin,
Zehong Shi, Jing Wang

Development of a Structure-from-Motion Pipeline

Entwicklung einer Structure-from-Motion Pipeline

In the field of computer vision, Structure-from-Motion (SfM) is a technique used to reconstruct 3D structures from sequences of 2D images, playing a critical role in applications like photogrammetry, 3D modeling, and autonomous navigation. The goal of this team project is to develop a complete SfM pipeline using open-source software and Python, which can reconstruct 3D structures from fisheye images taken from different viewpoints. For this, the project will use data from a fisheye camera array available at the LMS.

The project begins with a comprehensive literature review, where the team will investigate existing SfM methodologies, focusing on key topics such as feature detection, feature matching, camera pose estimation, and bundle adjustment. Additionally, the team should research fisheye camera calibration and its integration into SfM pipelines. Following the theoretical exploration, the team will gather data by capturing images from various viewpoints using fisheye cameras. Proper calibration of the cameras is essential to ensure accurate reconstruction results.

After collecting the dataset, the team will implement the SfM pipeline. This involves feature detection and matching between images, estimating camera poses, triangulating 3D points, and refining the model through bundle adjustment. Special attention will be given to ensuring the accuracy and robustness of the pipeline under varying conditions such as occlusions or lighting changes. The system will be evaluated using different datasets to assess reconstruction accuracy and computational efficiency.

Once the basic pipeline is in place, the team will extend it to work with data from a fisheye camera array available at the LMS. This step will require adapting the pipeline for the specific fisheye camera properties, particularly through specific calibration techniques and feature detection that account for lens distortions. The ultimate aim is to provide a flexible and modular SfM pipeline that can handle wide-angle camera setups, delivering accurate 3D reconstructions.

The project will be implemented in Python using open-source tools and relevant camera calibration libraries. The final deliverables include a fully functional SfM pipeline with comprehensive documentation, a dataset for testing the pipeline, a detailed evaluation report, and a demonstration of its application to fisheye cameras.

Begin: 01.10.2024
End: 28.02.2025

(PD Dr.-Ing. J. Seiler)

Declaration

We confirm that we have written this report independently, utilizing open-source libraries, and without using sources other than those listed. This report has never been submitted to another examination authority and accepted as part of an examination achievement, neither in this form nor in a similar form.

All content that was taken from a third party either verbatim or in substance has been acknowledged as such.

Place, Date

<full name>

Contents

Abstract	III
Symbols and Notations	IV
1 Introduction	1
2 Theoretical Foundations of the 3D Reconstruction Process	2
2.1 Fisheye Image Processing and Calibration	2
2.1.1 Fisheye Lens Distortion Model	2
2.1.2 Camera Calibration and Intrinsic Parameters	5
2.2 Structure-from-Motion (SfM) Pipeline	7
2.2.1 Camera Pose Estimation and 3D Reconstruction Using COLMAP	11
3 Implementation of the Fisheye-based SfM Pipeline	15
3.1 Feature Extraction and Matching	15
3.1.1 Capturing and Extracting Video Frames	15
3.1.2 Camera Calibrations	16
3.2 Feature Extraction and Matching	18
3.2.1 Feature Extraction	18
3.2.2 Feature Matching	19
3.3 Structure-from-Motion and Geometric Estimation	20
3.3.1 Estimating Camera Pose	21
3.4 3D Reconstruction: Triangulation and Bundle Adjustment	21
3.4.1 Refinement with Bundle Adjustment	22
3.4.2 Sparse 3D Reconstruction (Triangulation)	22
3.5 Dense 3D Reconstruction	23
3.5.1 Generating Dense Point Clouds	23
4 Results and Evaluation	24
4.1 Feature Matching Performance	24
4.2 3D Reconstruction Quality	26
4.2.1 Computational Efficiency	28

4.3 Impact of Scene Structure on Reconstruction Performance	28
5 Conclusion	31
List of Figures	33
List of Tables	34
Bibliography	35

Abstract

The rapid development of digital imaging and computer vision has made it possible to generate highly detailed 3D reconstructions from image data. Fisheye lenses, known for their wide field of view, also introduce unique distortion challenges. This project aims to apply the Structure-from-Motion (SfM) workflow to achieve 3D reconstruction from fisheye images. Our approach integrates several adaptations into the standard SfM workflow to handle the severe radial distortions characteristic of fisheye lenses. The project begins by capturing image data of target objects for 3D reconstruction using fisheye cameras, followed by implementing the checkerboard calibration algorithm to obtain camera parameters and correct lens distortions. We have optimized the feature detection algorithm to better accommodate the geometric distortions inherent in fisheye images, ensuring accurate and reliable feature matching. Additionally, the workflow incorporates a camera pose estimation algorithm tailored to the unique projection properties of fisheye lenses. To address the nonlinearities introduced by fisheye distortions, we have applied a robust triangulation method, making it more suitable for such cases. The core component of the optimized SfM workflow is Bundle Adjustment (BA), which has been meticulously refined to minimize projection errors across the entire image dataset, thereby optimizing the calibration and alignment of 3D structure points. To evaluate the performance of the developed SfM workflow, we conducted tests using COLMAP, where the extracted feature point data was stored. Compared to traditional methods, our approach demonstrated significant improvements in reconstruction accuracy and feature tracking stability across frames. The evaluation criteria for the test results included the accuracy of the 3D model and the robustness of feature tracking.

Symbols and Notations

This chapter lists the symbols and notations used throughout the document, presented in order of appearance to facilitate easy reference.

$\mathbf{x}$	Signal vector
N	Signal length
f	Focal length of the camera lens
R	Radial distortion coefficient
$\mathbf{K}$	Camera intrinsic matrix
$\mathbf{p}$	Homogeneous coordinates of a point in an image
$\mathbf{P}$	Homogeneous coordinates of a point in 3D space
I	Image intensity function
ω	Angular field of view of the fisheye lens
θ	Angle of incidence in the fisheye lens
$\mathbf{R}$	Rotation matrix
$\mathbf{t}$	Translation vector
$\mathbf{m}$	Image feature detected by the algorithm
σ	Standard deviation of noise
γ	Parameter for gamma correction in image processing
c	Speed of light in vacuum (for theoretical discussions)
$\mathbf{H}$	Homography matrix
π	Projection function mapping 3D points to 2D image plane
$\mathcal{L}$	Likelihood function used in optimization
$\mathbf{n}$	Normal vector to a surface in 3D space
$\mathbf{v}$	Velocity vector of a moving object
g	Acceleration due to gravity (if relevant to motion analysis)

This notation will be consistently used to discuss the mathematical modeling and analysis within the subsequent chapters, ensuring clarity and precision in the descriptions of the SfM processes and algorithms.

Chapter 1

Introduction

Three-dimensional reconstruction technology, by integrating multi-view 2D image information, provides crucial scene perception and modeling capabilities for applications such as robotic navigation, virtual reality, and digital cultural heritage preservation. While traditional multi-view reconstruction methods, such as binocular stereo vision, have reached a relatively mature stage, fisheye cameras offer unique advantages in reconstructing scenes within confined spaces or large-scale environments due to their ultra-wide field of view. The core objective of this project is to reconstruct 3D models of target objects from a set of images captured by a given fisheye camera. This is achieved by systematically integrating existing open-source tools and calibration methods into a fisheye-compatible 3D reconstruction workflow. Specifically, we leverage computer vision libraries within the Python ecosystem (OpenCV) and incorporate fisheye camera calibration techniques to optimize key processes, including image preprocessing, feature matching, and bundle adjustment. By utilizing existing open-source 3D reconstruction tools, such as COLMAP, we aim to adapt these steps for distortion-corrected fisheye images, ensuring the recovery of high-precision 3D structures. The remainder of this report is structured as follows: Chapter 2 provides an overview of the algorithmic theories involved in the reconstruction process. Chapter 3 details the implementation of these algorithms in image processing, feature extraction, parameter configuration, and data handling. Chapter 4 evaluates reconstruction accuracy and efficiency based on image matching consistency and the quality of the reconstructed model. Chapter 5 summarizes the technical limitations encountered in the task and the key findings achieved.

Chapter 2

Theoretical Foundations of the 3D Reconstruction Process

2.1 Fisheye Image Processing and Calibration

We first acquired a video of the target object captured by a fisheye camera. By extracting frames from the video, we obtained several sets of fisheye images of the target object. However, since images captured by a fisheye camera inherently exhibit distortion, it is necessary to perform distortion correction prior to 3D reconstruction. This step ensures the removal of distortions and generates rectified frames suitable for feature extraction. The following diagram presents the flowchart of our task.

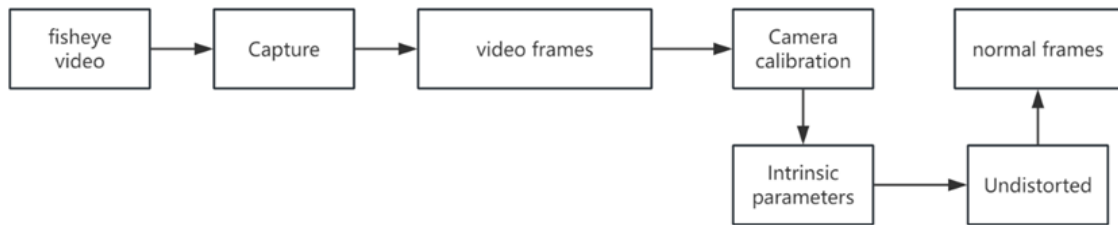


Figure 2.1: Workflow of fisheye image processing and calibration.

2.1.1 Fisheye Lens Distortion Model

Characteristics of Fisheye Cameras : A fisheye camera is a type of wide-angle lens that features an extremely large field of view, enabling it to capture an exceptionally broad scene, sometimes even representing the entire scene in a circular or hemispherical form. One of the key characteristics of fisheye cameras is their ultra-wide field of view. However, due to wide-angle distortion, the periphery of the image may exhibit stretching or warping effects.

Fisheye cameras are commonly used in photography and video applications that require capturing expansive scenes.

The schematic diagram of the incidence and reflection of the fisheye camera is shown below.

For a point $P(x, y, z)$ in the camera coordinate system, if it is projected according to the pinhole camera model, there is no distortion, and the image point is $P_0(a, b)$. After distortion occurs, the distorted image point coordinates are denoted as p' . In the diagram, $r = ||OP_0||$ represents the undistorted radius, and $r_d = ||OP'||$ represents the distorted radius.

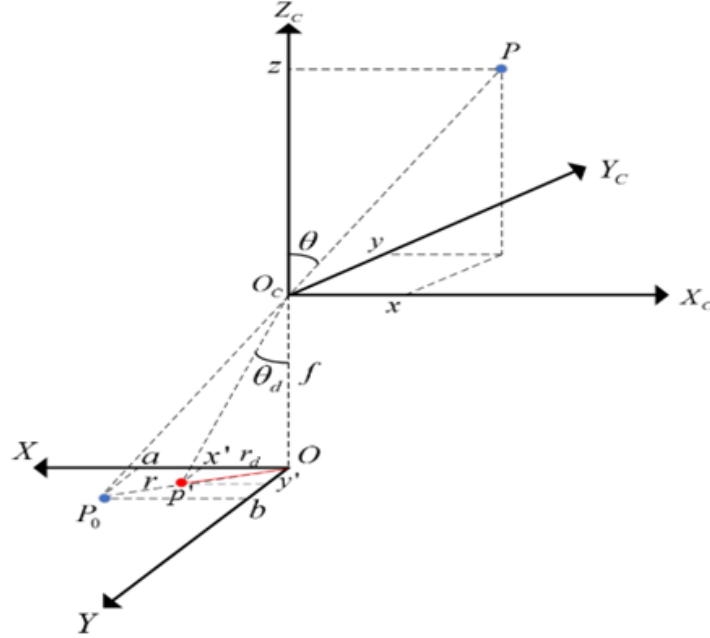


Figure 2.2: Schematic diagram of fisheye lens distortion.

Distortion Correction Model for Fisheye Cameras : In the schematic diagram above, we assume $f = 1$ without loss of generality. This allows us to determine the ratio between r_d and r , which is independent of f . Consequently, the undistorted image point coordinates $P_0(a, b)$ and the incidence angle θ can be obtained. Here, $(a, b, 1)$ represents the homogeneous coordinates corresponding to P_0 . In practice, due to various factors, a real fisheye lens does not precisely conform to the ideal projection model. To facilitate fisheye camera calibration, it is common to approximate the actual projection function of the fisheye lens using the first five terms of the Taylor expansion of r_d with respect to $\theta[1]$.

The model, which is widely used in OpenCV and other general applications, represents the fisheye projection function as an odd-order Taylor series expansion of the incidence angle θ :

$$r_d = \theta(k_0 + k_1\theta^2 + k_2\theta^4 + k_3\theta^6 + k_4\theta^8)$$

Typically, we set $k_0 = 1$ so that the distortion effects are reflected in the higher-order terms involving k_1, k_2, k_3, k_4 , leading to the refined approximation:

$$r_d = \theta(1 + k_1\theta^2 + k_2\theta^4 + k_3\theta^6 + k_4\theta^8)$$

By considering the normalized camera coordinates of the distorted point (x', y') , the undistorted coordinates (a, b) can be derived as follows:

$$u = fx' + c_x, \quad v = fy' + c_y$$

$$\left(\frac{x}{r}\right) = \left(\frac{a}{r_d}\right), \quad \left(\frac{y}{r}\right) = \left(\frac{b}{r_d}\right)$$

It is important to note that $r_d \neq \theta_d$. According to the schematic diagram, both the general fisheye model and the equidistant projection model satisfy the following strict relationships:

$$\tan(\theta) = \frac{OP_r}{r}$$

For $f = 1$, this simplifies to:

$$\theta = \arctan(r) \quad \text{or} \quad \theta = \arctan(r_d)$$

Numerical Solution for Distortion Correction : In practical computations, the goal is to determine the undistorted pixel coordinates (u, v) from a given undistorted pixel position (i, j) . This is achieved by using the remap function, which interpolates pixel values accordingly. Conversely, when we need to map a known undistorted pixel coordinate (u, v) back to an undistorted pixel coordinate, we must first determine (x', y') . Given that r_d is known, we must solve for the corresponding θ .

Thus, the core problem of distortion correction is solving the following nonlinear polynomial equation for θ :

$$r_d = \theta(1 + k_1\theta^2 + k_2\theta^4 + k_3\theta^6 + k_4\theta^8)$$

For solving nonlinear equations, common methods include bisection methods, fixed-point iteration, and Newton's iterative method. Here, Newton's method is employed for its efficiency in finding numerical solutions.

Define the function:

$$f(\theta) = \theta + k_1\theta^3 + k_2\theta^5 + k_3\theta^7 + k_4\theta^9 - r_d$$

Using the initial estimate $\theta_0 = r_d$, the iterative update formula is given by:

$$\theta_{n+1} = \theta_n - \frac{f(\theta_n)}{f'(\theta_n)}$$

The iteration continues until $f(\theta) \approx 0$, subject to a predefined convergence threshold (e.g., $1e^{-6}$) or a maximum iteration limit. Once θ is obtained, the undistorted image coordinates $P_0(a, b)$ are computed as:

$$a = \frac{r}{r_d}x'$$

$$b = \frac{r}{r_d}y'$$

This approach enables precise distortion correction, making it a widely adopted method in fisheye image processing.

2.1.2 Camera Calibration and Intrinsic Parameters

To determine the spatial geometric position of a point on the surface of an object and its corresponding point in the image, it is necessary to establish a geometric model of camera imaging. The parameters of these geometric models are the camera parameters. In most cases, these parameters must be obtained through experiments and computations. The process of solving for these parameters is known as camera calibration. Since we use a fisheye camera to capture the target object and perform 3D reconstruction, it is necessary to calibrate the camera, which involves calculating its intrinsic parameters. In this task, we employed the chessboard calibration method [2] to determine the camera's intrinsic parameters.

Composition of Intrinsic and Extrinsic Matrices in Camera Calibration : In camera calibration, the intrinsic parameters (5 variables) include focal length f , principal point coordinates (u_0, v_0) , and distortion parameters.

The intrinsic matrix $\mathbf{A}$ represents parameters that are inherent to the camera itself

and remain unchanged regardless of the camera's position or orientation. It is expressed as:

$$\mathbf{A} = \begin{bmatrix} f_x & \gamma & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

To solve for the intrinsic matrix $\mathbf{A}$, we follow the calculation below: The transformation from world coordinates to image coordinates follows the homogeneous projection equation: $x = MX$ where $\mathbf{M}$ is the homogeneous projection matrix. From the relation:

$$\mathbf{M} = \mathbf{A}[\mathbf{R}_1, \mathbf{R}_2, \mathbf{T}]$$

we aim to extract $\mathbf{A}$.

Since $\mathbf{R}_1, \mathbf{R}_2$ are the first two columns of a rotation matrix, they must satisfy the following orthogonality constraints:

$$\mathbf{R}_1^T \mathbf{R}_2 = 0, \quad \mathbf{R}_1^T \mathbf{R}_1 = \mathbf{R}_2^T \mathbf{R}_2 = 1.$$

Rewriting:

$$\mathbf{R}_1 = \mathbf{A}^{-1} \mathbf{M}_1, \quad \mathbf{R}_2 = \mathbf{A}^{-1} \mathbf{M}_2$$

Multiplying both sides:

$$\mathbf{M}_1^T \mathbf{A}^{-T} \mathbf{A}^{-1} \mathbf{M}_2 = 0, \quad \mathbf{M}_1^T \mathbf{A}^{-T} \mathbf{A}^{-1} \mathbf{M}_1 = \mathbf{M}_2^T \mathbf{A}^{-T} \mathbf{A}^{-1} \mathbf{M}_2 = 1.$$

Define:

$$\mathbf{B} = \mathbf{A}^{-T} \mathbf{A}^{-1}.$$

Since $\mathbf{B}$ is a symmetric matrix, it has the form:

$$\mathbf{B} = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \end{bmatrix}$$

Each calibration board image provides one constraint equation:

$$\mathbf{M}_i^T \mathbf{B} \mathbf{M}_j = v_{ij} b$$

where:

$$b = [B_{11}, B_{12}, B_{22}, B_{13}, B_{23}, B_{33}]^T$$

Each image provides two constraint equations, and since $\mathbf{B}$ has 6 unknown elements, at least 3 images are needed to solve for b . When more than 3 images are used, the least squares method is applied to find the best estimate of $\mathbf{B}$, which then determines the intrinsic matrix $\mathbf{A}$.

Extracting intrinsic parameters from $\mathbf{B}$:

$$v_0 = \frac{B_{12}B_{13} - B_{11}B_{23}}{B_{11}B_{22} - B_{12}^2}, \quad \beta = \sqrt{\frac{B_{11}}{B_{11}B_{22} - B_{12}^2}}, \quad \gamma = -B_{13}\alpha^2\beta, \quad u_0 = \frac{\gamma v_0}{\beta} - B_{13}\alpha^2.$$

This step recovers the intrinsic camera matrix $\mathbf{A}$ from the computed $\mathbf{B}$.

2.2 Structure-from-Motion (SfM) Pipeline

Structure from Motion (SfM) typically begins with feature extraction and matching, followed by geometric verification to filter out outliers. After these steps, a scene graph is obtained, which serves as the foundation for the subsequent incremental reconstruction. In incremental reconstruction, two frames are selected for the initial reconstruction. Before determining the camera pose of the current frame within the map, it is necessary to perform triangulation to compute 3D scene points, filter out outliers, and apply Bundle Adjustment (BA) to optimize the current model.

Under the Structure-from-Motion (SfM) framework, our task is completed according to the following workflow diagram.

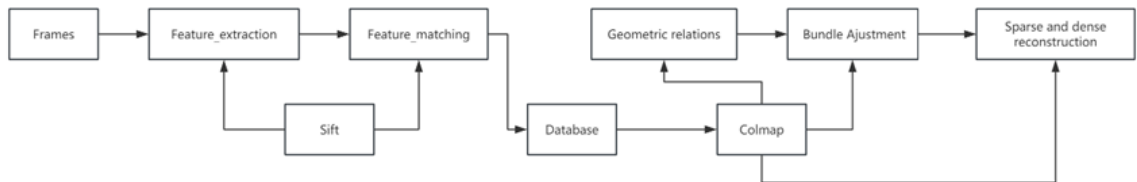


Figure 2.3: Structure-from-Motion (SfM) workflow.

Feature Extraction (SIFT Algorithm)

In this work, we utilized the SIFT (Scale-Invariant Feature Transform) algorithm [3] to extract features from the acquired images. SIFT consists of both an interest point detector and a descriptor. The SIFT descriptor is highly robust and is often combined with various interest point detectors. SIFT features exhibit scale, rotation, and illumination invariance, making them well-suited for reliable matching under different 3D perspectives and noise conditions.

The SIFT descriptor, denoted as $h(x, y, \theta)$, represents the statistical results of Gaussian image gradients in the neighborhood of a feature point. It is a three-dimensional array but is typically represented as a vector. This vector is obtained by arranging the 3D array in a specific order. Since the feature descriptor is dependent on the scale at which the feature point is located, gradient calculations should be performed on the Gaussian image corresponding to the detected feature point.

The following defines the Descriptor Sampling Region. The SIFT descriptor $h(x, y, \theta)$ represents the statistical results of Gaussian image gradients in the neighborhood of a feature point. The neighborhood around a feature point is divided into $B_p \times B_p$ sub-regions, where each subregion has a size of $m\sigma$ pixels, with $m = 3$ and $B_p = 4$. Here, σ represents the scale of the feature point. Considering practical calculations, bilinear interpolation is required, and thus the image region used for computation extends to $m\sigma(B_p + 1)$. If rotation factors are also taken into account, the actual computation region should be further extended to $m\sigma(B_p + 1)\sqrt{2}$.

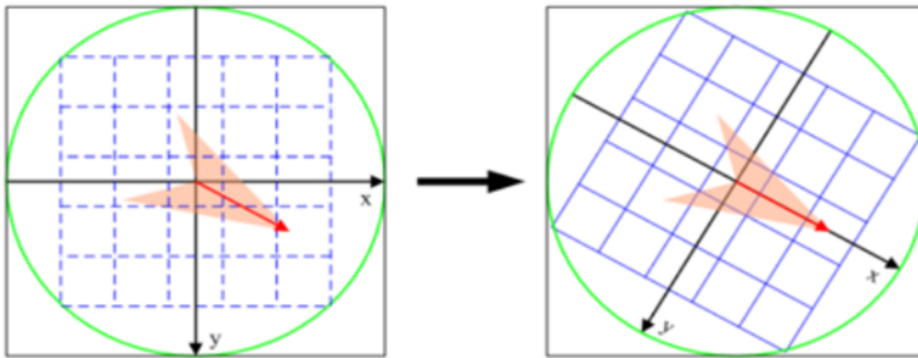


Figure 2.4: Feature extraction and descriptor sampling region in SIFT.

Rotating the Image to the Main Orientation : To ensure that the feature vector maintains rotation invariance, the image gradient positions and directions in the neigh-

borhood of the feature point need to be rotated to align with the main orientation. Specifically, the region $m\sigma(B_p + 1)\sqrt{2} \times m\sigma(B_p + 1)\sqrt{2}$ centered around the feature point is rotated by an angle θ , aligning the original image's x-axis with the dominant direction. The rotation transformation follows this formula:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

After rotating the gradient positions and directions in the feature point's neighborhood, a $m\sigma B_p \times m\sigma B_p$ image region is extracted from the rotated image, centered at the feature point. This region is then evenly divided into $B_p \times B_p$ image regions. Each of these regions is divided into $B_p \times B_p$ subregions, with each interval being $m\sigma$ pixels apart.

Generating the Feature Vector : In each subregion, an 8-bin gradient orientation histogram is computed by accumulating the gradient magnitudes along different orientations. Unlike the method used for determining the main feature direction, the gradient orientation histogram divides the $0^\circ - 360^\circ$ range into 8 discrete bins, each spanning 45° . This results in each seed point containing 8 gradient intensity values corresponding to these directions.

Since the descriptor divides the region into $4 \times 4(B_p \times B_p)$ subregions, the total number of gradient intensity values is:

$$4 \times 4 \times 8 = 128$$

Thus, the final SIFT feature vector is a 128-dimensional descriptor, which encapsulates robust local image information, making it highly distinctive and invariant to scale, rotation, and illumination changes.

Normalization of the Feature Vector : To eliminate the impact of illumination variations, the generated 128-dimensional SIFT feature vector needs to be normalized. Normalization ensures that the descriptor remains invariant to changes in brightness and contrast.

Feature Matching and Correspondences (Feature Matching and RANSAC)

The RANSAC (Random Sample Consensus) [4] algorithm can estimate more accurate model parameters from image matching data containing outliers through an iterative approach. Compared to the least squares method, RANSAC is more effective in eliminating mismatches, especially when the dataset contains a high proportion of outliers.

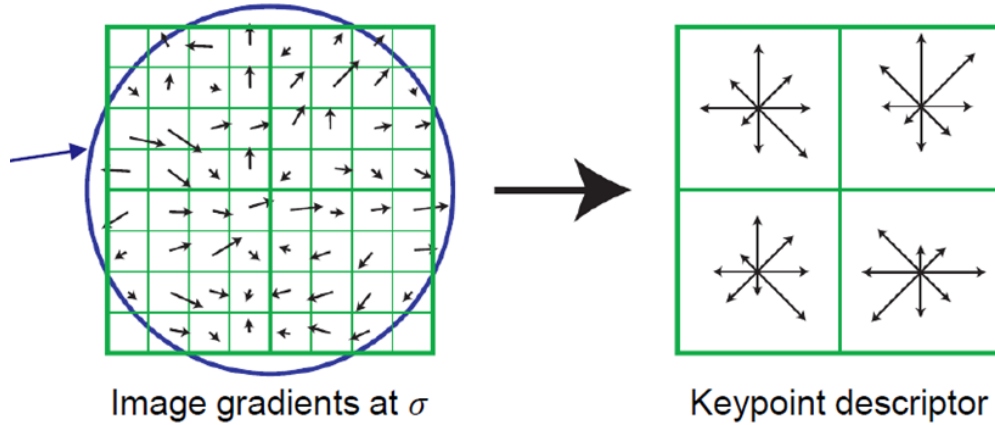


Figure 2.5: Gradient computation and SIFT keypoint descriptor representation.

Its working principle involves randomly selecting a sample, computing the model, and then evaluating the number of inliers and projection error in the sample set. By iterating, it finds the optimal model. Although RANSAC has the drawbacks of an indefinite number of iterations and the possibility of obtaining a suboptimal result, it demonstrates robustness in applications such as image stitching. Therefore, in this task, we have chosen RANSAC for feature matching.

RANSAC (Random Sample Consensus) is an iterative method used to estimate a mathematical model from a set of observed data while eliminating outliers. It repeatedly selects random subsets of the data to find the best-fitting model.

In OpenCV, RANSAC is commonly used to filter out incorrect feature matches and estimate the best homography matrix H , which is a 3×3 transformation matrix. The goal of RANSAC is to find the optimal transformation matrix where the number of inliers (points that fit the model) is maximized. Typically, the matrix is normalized by setting $h_{33} = 1$. Since the homography matrix has 8 unknown parameters, at least 8 linear equations are required for solving it. Each point correspondence provides two equations, meaning a minimum of 4 matched point pairs is needed.

The homography transformation is defined as:

$$s \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

where (x, y) represents the keypoint in the target image. (x', y') represents the corresponding keypoint in the scene image. s is the scale parameter.

2.2.1 Camera Pose Estimation and 3D Reconstruction Using COLMAP

This task is based on the existing COLMAP code and database, focusing on processing the undistorted images and feature points obtained from the target. Therefore, the following sections primarily introduce the principles of triangulation and bundle adjustment (BA) optimization methods. Here we introduce the Stereo Triangulation method [5]: In SLAM (Simultaneous Localization and Mapping), triangulation is a fundamental step for estimating the 3D coordinates of newly detected feature points in the environment. This process recovers 3D points from two viewpoints and, when multiple views are available, it enables a more accurate reconstruction of 3D structures.

Given two matched feature points in different images and the relative rotation and translation $[R|t]$ between the two camera views, our goal is to compute the 3D coordinates of the corresponding feature points.

In SLAM (Simultaneous Localization and Mapping), triangulation is a fundamental step for estimating the 3D coordinates of newly detected feature points in the environment. This process recovers 3D points from two viewpoints and, when multiple views are available, it enables a more accurate reconstruction of 3D structures. We set the input as: corresponding matched points from two images (usually in normalized coordinates) and relative rotation and translation $[R|t]$ between the two images.

Meanwhile, the output is the 3D coordinates of the feature point. Assume P is a 3D point in the world coordinate system, with coordinates:

$$X = [X, Y, Z]^T$$

Its projections onto two different views have the following image coordinates: x_1, x_2 (These are in normalized coordinates, meaning they have been mapped to a canonical camera coordinate system.) The relationship between the 3D point and its projections can be described by the camera projection model, and we aim to solve for X . During our task, we use the DLT (Direct Linear Transform) [6] method to maintain robust and efficient triangulation.

The Direct Linear Transform (DLT) method uses projection matrices to establish a linear system for solving the 3D coordinates of a point. For a given 3D point X in homogeneous coordinates, its projections onto two images are:

$$p_1 = \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} = \begin{bmatrix} X_1/Z_1 \\ Y_1/Z_1 \\ 1 \end{bmatrix}, \quad p_2 = \begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} X_2/Z_2 \\ Y_2/Z_2 \\ 1 \end{bmatrix}$$

where P_1 and P_2 are the camera projection matrices for two views:

$$x_1 = P_1 X, \quad P_1 = K[I|0], \quad x_2 = P_2 X, \quad P_2 = K[R|t]$$

To eliminate the unknown scale factor, we cross-multiply both sides by x_1 :

$$x_1 \times (x_1 P_1 X) = 0, \quad \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \times P_1 X = \begin{bmatrix} 0 & -1 & y_1 \\ 1 & 0 & -x_1 \\ -y_1 & x_1 & 0 \end{bmatrix} P_1 X = 0.$$

Rewriting in terms of the matrix elements:

$$\begin{bmatrix} 0 & -1 & y_1 \\ 1 & 0 & -x_1 \\ -y_1 & x_1 & 0 \end{bmatrix} \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} X = 0.$$

Expanding, we obtain:

$$\begin{bmatrix} -p_{21} + y_1 p_{31} & -p_{22} + y_1 p_{32} & -p_{23} + y_1 p_{33} & -p_{24} + y_1 p_{34} \\ p_{11} - x_1 p_{31} & p_{12} - x_1 p_{32} & p_{13} - x_1 p_{33} & p_{14} - x_1 p_{34} \end{bmatrix} X = 0.$$

Solving for X Using Singular Value Decomposition (SVD) : Each camera view provides two constraints (one for x_1 and one for y_1), and combining the constraints for two views, we obtain:

$$\begin{bmatrix} x_1 P_1 \text{row}(2) - P_1 \text{row}(0) \\ y_1 P_1 \text{row}(2) - P_1 \text{row}(1) \\ x_2 P_2 \text{row}(2) - P_2 \text{row}(0) \\ y_2 P_2 \text{row}(2) - P_2 \text{row}(1) \end{bmatrix} X = 0.$$

This forms a homogeneous linear system: $AX = 0$. The solution for X is obtained by performing Singular Value Decomposition (SVD) on matrix A. The smallest singular value's corresponding singular vector gives the solution for X in homogeneous coordinates:

$$X = [X, Y, Z, W]^T.$$

To obtain the final 3D point coordinates, we normalize:

$$X' = \begin{bmatrix} X/W \\ Y/W \\ Z/W \end{bmatrix}^T.$$

At the same time, we also apply incremental SfM: Prior to this, we have already obtained the transformation matrix (rotation matrix and translation vector) between two cameras, which can be achieved by estimating the essential matrix and decomposing it. Now, let camera 1's coordinate system be the world coordinate system. When adding a third view, the transformation matrix from the third camera (hereafter referred to as camera 3) to camera 1's coordinate system is determined using the binocular reconstruction method. That is, feature points are extracted and matched between the third image and the first image, followed by triangulation-based reconstruction. On this basis, the fourth, fifth, and subsequent images are added. As the number of images increases, the difference between newly added images and the first image may become increasingly large, making feature point extraction extremely difficult. At this point, the binocular reconstruction method can no longer be used. Instead, we adopt the PnP method, which determines the camera's position and pose in space based on the correspondence between 3D points in space and 2D points in the image. In other words, if we know some 3D spatial points in camera 1's coordinate system and also know the corresponding 2D points in camera 3's image, PnP can be used to solve for the transformation matrix (rotation matrix and translation vector) between camera 3 and camera 1. A general process of multi-view 3D reconstruction: The binocular reconstruction method is used to reconstruct the first two images, thereby obtaining some spatial points. After adding the third image, feature matching is performed between it and the second image. Among these matched points, some must also be points that were matched between image 1 and image 2. In other words, some of these matching points already have known spatial coordinates in camera 1's coordinate system, and their 2D pixel coordinates in the third image are also known. All the necessary information for solving PnP is now available. Since the spatial point coordinates are all in camera 1's coordinate system, the camera position and pose solved by PnP are also in camera 1's coordinate system, meaning the transformation matrix from camera 3 to camera 1 is obtained.

Using the transformation matrix between camera 3 and camera 1, more point clouds are reconstructed via triangulation and integrated into the existing point cloud.

At the same time, BA optimization[6] is also performed in COLMAP. To elimi-

nate accumulated errors, BA optimization is required after image registration and triangulation. Since incremental reconstruction only affects neighboring frames, it is unnecessary to perform GBA every time. Therefore, after an image is registered, LBA is performed only on the most relevant set of images, while GBA is only applied when the model grows to a certain extent, helping distribute the computational cost. In LBA, the Cauchy kernel function is used to handle outliers, and when optimizing with hundreds of cameras, the PCG solver is used. After BA, as the camera poses and map points are updated, many observations may no longer meet the required accuracy, so observations with large reprojection errors are removed, geometric verification conditions are checked, and for GBA, camera degradation is also examined. Since BA optimizes camera poses, points that could not be triangulated before due to inaccurate or incorrect poses may now become triangulatable, making it necessary to perform a post-triangulation step after BA, similar to the triangulation process before BA. Due to the influence of outliers, a single BA iteration often results in a significant number of points being filtered out, and after post RT, new map points are introduced to expand the reconstruction.

Chapter 3

Implementation of the Fisheye-based SfM Pipeline

3.1 Feature Extraction and Matching

The raw fisheye images are obtained through video recording, ensuring frame continuity and sufficient viewpoint variation, which is crucial for accurate 3D reconstruction. The recorded video is then processed to extract individual frames, which serve as the primary dataset for subsequent processing.

3.1.1 Capturing and Extracting Video Frames

Based on this code, we extract video frames at intervals. The image below is one of the extracted frames. Code:

```
if frame_count % interval == 0:
    output_name = os.path.join(output_folder,
                                f"frame_{save_count}.jpg")
```

OpenCV (**Open Source** Computer Vision Library) is an open-source library that provides tools for real-time computer vision, image processing, and machine learning, widely used in robotics, AI, and multimedia applications.

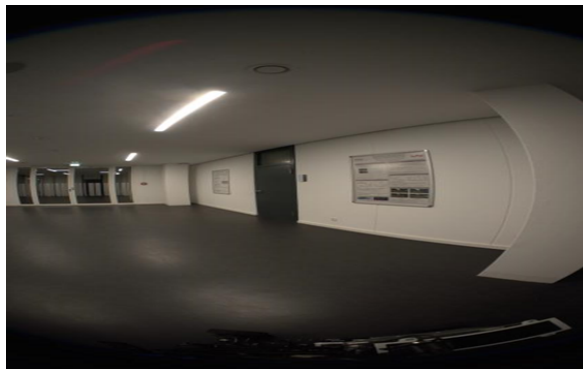


Figure 3.1: Extracted video frame from fisheye video input.

3.1.2 Camera Calibrations

Based on this code, we define the checkerboard size, termination criteria, and calibration flags. The specific method for camera calibration using the checkerboard approach has been explained in detail in Chapter 2. Code:

```
CHECKERBOARD = (9,7)
subpix_criteria = (cv2.TERM_CRITERIA_EPS +
    cv2.TERM_CRITERIA_MAX_ITER, 300, 0.01)
calibration_flags = cv2.fisheye.CALIB_RECOMPUTE_EXTRINSIC +
    cv2.fisheye.CALIB_FIX_SKEW
```

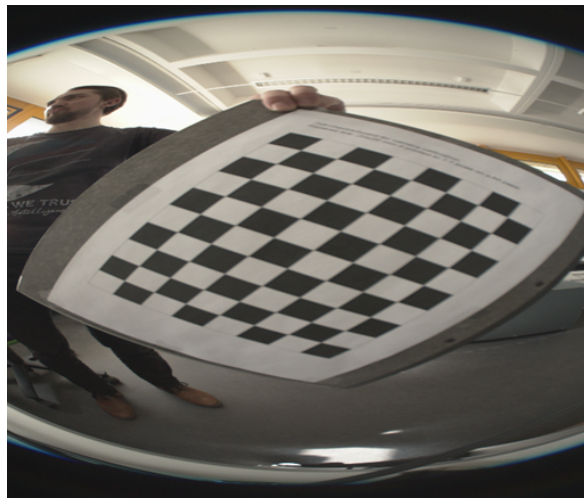


Figure 3.2: Fisheye calibration image using checkerboard pattern.

The camera calibration process using a checkerboard involves several key steps. First, calibration images are retrieved using `glob.glob`. Each image is converted to grayscale, and `cv2.findChessboardCorners` is used to detect checkerboard corners. The image below is the calibrated result after applying the camera calibration process.

If corners are found, they are refined using `cv2.cornerSubPix`, and corresponding 3D object points (`objp`) and 2D image points are stored. To verify detection, `cv2.drawChessboardCorners` is used to visualize the detected corners, and annotated images are saved. Finally, camera calibration is performed using `cv2.fisheye.calibrate`, extracting the intrinsic matrix $\mathbf{K}$.

$$K = np.array \left(\begin{bmatrix} 591.0412707624 & 0.0 & 929.1522869318215 \\ 0.0 & 594.439320988173 & 535.87156962935 \\ 0.0 & 0.0 & 1.0 \end{bmatrix} \right)$$

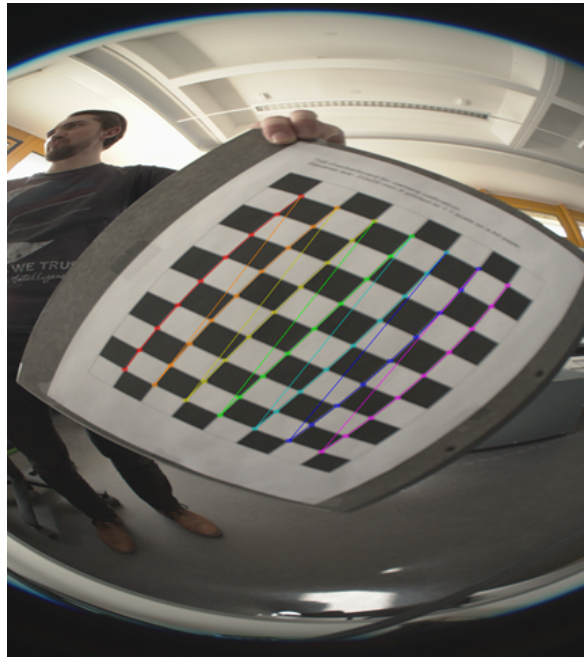


Figure 3.3: Chessboard calibration visualization.

This concludes the steps for feature extraction, frame selection, and camera calibration in the fisheye-based SfM pipeline.

Image Undistortion

Definition of the Undistortion Function we use the code:

```
def undistort(image, balance=0.0, dim2=None, dim3=None):
    dim1 = image.shape[:2][::-1] # Get input image dimensions
    (width, height)
    assert dim1[0] / dim1[1] == DIM[0] / DIM[1], "Image aspect ratio
        does not match."

    # Scale dimensions and intrinsic parameters
    dim1 = image.shape[:2][::-1]
    scaled_K = K * dim1[0] / DIM[0]
    scaled_K[2][2] = 1.0

    # Estimate the new camera matrix
    new_K = cv2.fisheye.estimateNewCameraMatrixForUndistortRectify(
        scaled_K, D, dim2, np.eye(3), balance=balance, fov_scale=0.8
    )

    # Generate the undistortion mapping
```

```
map1, map2 = cv2.fisheye.initUndistortRectifyMap(  
    scaled_K, D, np.eye(3), new_K, dim3, cv2.CV_16SC2  
)  
  
# Apply undistortion using the mapping  
undistorted_img = cv2.remap(image, map1, map2,  
    interpolation=cv2.INTER_LINEAR,  
    borderMode=cv2.BORDER_CONSTANT)
```

The image below is the result of distortion correction using the intrinsic matrix $\mathbf{K}$.

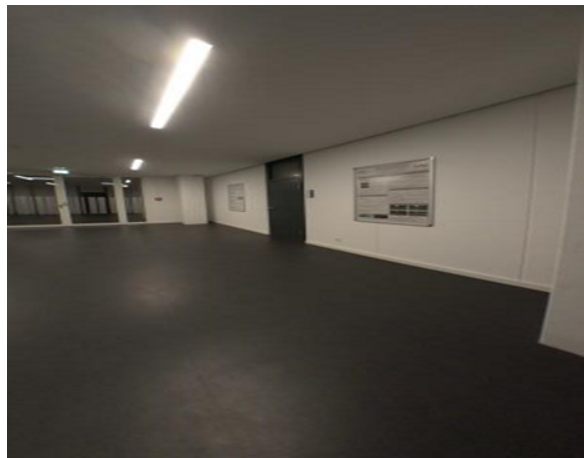


Figure 3.4: Distortion correction result

3.2 Feature Extraction and Matching

We extracted keypoints from the target image using the SIFT (Scale-Invariant Feature Transform) algorithm.

3.2.1 Feature Extraction

We defined and extracted feature points using the SIFT library with the following code:

```
sift = cv2.SIFT_create()  
keypoints1, descriptors1 = sift.detectAndCompute(gray1, None)
```

The image below shows the extracted keypoints obtained using the SIFT algorithm.



Figure 3.5: Extracted keypoints using SIFT

3.2.2 Feature Matching

We performed feature matching using the FLANN-based matcher, where matches were filtered using Lowe’s Ratio Test by retaining only those with a distance ratio less than 0.6 to ensure high-quality correspondences. Specifically, the best match (m) and second-best match (n) were compared, keeping only those where:

$$m.distance < 0.6 * n.distance$$

To further refine the matches, RANSAC was applied by extracting the matched point coordinates and computing the fundamental matrix using:

```
cv.findFundamentalMat
```

Filtering outliers based on the returned mask to obtain the final:

```
ransac_matches
```

The image below shows the feature matching results after RANSAC processing.

Storing Matches in a Database

The COLMAP database organizes image and feature data into structured tables for 3D reconstruction. The cameras table stores camera parameters such as model type, image size, focal length, and principal point. The images table records image-related details, including the filename, corresponding camera ID, and extrinsic parameters like rotation and translation. Feature point coordinates are saved in the *keypoints* table, while their corresponding descriptors are stored in the descriptors table for matching. Initial feature matches between image pairs are recorded in the *matches* table. To improve reliability, geometrically verified matches, such as the fundamental matrix



Figure 3.6: Feature matching results after RANSAC

(F) and essential matrix (E), are stored in the *two_view_geometries* table.

To insert this data into the database, we connect to COLMAP using:

```
db = COLMAPDatabase.connect(database_path)
```

Then execute an SQL statement to insert feature matches and geometric relationships into *two_view_geometries*:

```
db.execute(  
    """INSERT INTO two_view_geometries  
    (pair_id, rows, cols, data, config, F, E, H, qvec, tvec)  
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)"""  
)
```

3.3 Structure-from-Motion and Geometric Estimation

We estimated the camera poses using COLMAP, which computes the camera's position and orientation through Structure-from-Motion (SfM). COLMAP first detects and matches feature points across images, then refines these correspondences using geometric verification. The essential matrix (E) or fundamental matrix (F) is computed to establish relative camera poses between image pairs. With an initial set of poses, COLMAP performs incremental reconstruction, optimizing the extrinsic parameters

(rotation and translation) through bundle adjustment to refine camera positions. This process results in accurate pose estimation, which is essential for 3D reconstruction.

3.3.1 Estimating Camera Pose

The initial image pair estimation in COLMAP begins by selecting a reliable image pair with sufficient feature correspondences and well-conditioned geometry using:

```
mapper.register_initial_image_pair(mapper_options ,
    two_view_geometry, init_pair)
```

This establishes the initial camera pose as the foundation for reconstruction.

For subsequent image registration, COLMAP employs an incremental Structure-from-Motion (**SfM**) approach, iteratively adding images while refining camera poses and the 3D structure through:

```
reg_next_success = mapper.register_next_image(mapper_options ,
    next_image_id)
```

Since the dataset consists of many consecutive frames, this method ensures a robust and accurate camera pose estimation, with theoretical details of incremental **SfM** discussed in the relevant section.

3.4 3D Reconstruction: Triangulation and Bundle Adjustment

In COLMAP, robust and efficient triangulation is performed using the Direct Linear Transform (DLT) method, which reconstructs 3D points from multiple image observations. The theoretical principles of DLT have been discussed in the relevant section. To further refine the reconstruction, Bundle Adjustment (BA) is applied, optimizing camera poses and 3D point positions by minimizing reprojection errors. This combination of DLT-based triangulation and BA optimization ensures accurate and stable 3D reconstruction.

In COLMAP, triangulation and Bundle Adjustment (BA) are essential for refining 3D reconstruction. Triangulation is performed using:

```
mapper.triangulate_image(options.get_triangulation(), next_image_id)
```

which incorporates observations from newly registered images to estimate 3D point positions. To further optimize the reconstruction, BA is applied through:

```
bundle_adjuster =
    pycolmap.create_default_bundle_adjuster(ba_options, ba_config,
    reconstruction)
summary = bundle_adjuster.solve()
```

which adjusts camera parameters and 3D points by minimizing reprojection errors. Local BA focuses on a target image and its neighboring images, optimizing newly added points, especially short track 3D points, for better local accuracy. In contrast, Global BA refines the entire reconstruction by including all registered images while fixing the first two images' poses to maintain geometric stability. This iterative process ensures robust and precise 3D structure estimation for an accurate reconstruction.

3.4.1 Refinement with Bundle Adjustment

In COLMAP, iterative refinement is applied in both local and global Bundle Adjustment (BA) to enhance reconstruction accuracy. Local BA iteratively adjusts the camera parameters and 3D points around a target image, refining them over multiple iterations. After each optimization step, the change ratio of adjusted observations is checked, and if the variation is minimal, the process stops. To accelerate convergence, the loss function can be adjusted dynamically.

In global BA, refinement starts with track merging and re-triangulation of 3D points, followed by repeated BA optimization. After each iteration, the change ratio of observation values is evaluated, and if it falls below a predefined threshold, convergence is assumed, and further iterations are halted. The optimization loop executes multiple rounds of global or local BA, as shown in:

```
for _ in range(max_num_refinements):
    run_global_ba(mapper, mapper_options, ba_options) # Run Global
    BA
    report = run_local_ba(mapper, mapper_options, ba_options,
        tri_options,
        image_id, point3D_ids) # Run Local BA
    if change_ratio < max_refinement_change:
        break
```

This iterative BA refinement ensures stable and precise camera pose estimation and 3D structure reconstruction, leading to a more accurate final model.

3.4.2 Sparse 3D Reconstruction (Triangulation)

To process 3D point data in COLMAP, the binary 3D point file is first read using:

```
rw.read_points3D_binary(points3d_file)
```

extracting all reconstructed points and counting them to confirm successful loading. Next, the (x, y, z) coordinates of each point are extracted, stored in a list, and converted into a NumPy array for efficient processing. Finally, an Open3D point cloud object is created, and the extracted coordinates are assigned to it, forming a structured dataset suitable for visualization and further analysis.

3.5 Dense 3D Reconstruction

3.5.1 Generating Dense Point Clouds

In COLMAP, Patch Match Stereo and Stereo Fusion are used for dense 3D reconstruction. First, the `patch_match_stereo` command is executed on undistorted images to perform dense stereo matching, estimating depth maps for each pixel. To improve accuracy, geometric consistency checking is enabled (`-PatchMatchStereo.geom_consistency true`), and GPU acceleration is specified (`-PatchMatchStereo.gpu_index 0`).

Next, the `stereo_fusion` command merges multiple depth maps into a single dense point cloud, producing an output file (typically `fused.ply`). Key parameters include `-input_type` for selecting geometric information as input and `-StereoFusion.num_threads` to enable multi-threaded processing for efficiency.

The entire pipeline is executed using the `run_colmap_command` function, which sequentially calls COLMAP command-line tools, ultimately generating a dense point cloud (PLY file) for high-resolution 3D reconstruction.

Chapter 4

Results and Evaluation

This chapter presents a comprehensive evaluation of our fisheye-adapted Structure-from-Motion (SfM) pipeline. The assessment focuses on three main aspects: feature matching performance, 3D reconstruction quality, and a comparative study between indoor and outdoor reconstruction results.

The evaluation methodology includes both qualitative and quantitative analyses. We examine the effectiveness of distortion correction in feature matching by comparing keypoint distributions before and after undistortion. For 3D reconstruction, we assess the geometric consistency and completeness of both sparse and dense reconstructions in different environments. Finally, we compare indoor and outdoor reconstruction results to highlight the impact of scene complexity and environmental conditions.

In addition, we discuss the limitations of the current approach and propose potential improvements to enhance reconstruction accuracy and computational efficiency in fisheye-based SfM applications.

4.1 Feature Matching Performance

SIFT remains a widely used feature detector due to its robustness to scale and rotation variations, making it suitable for fisheye imagery despite radial distortions [7]. Distortion correction is a crucial preprocessing step in fisheye SfM, as it directly affects keypoint stability and feature matching accuracy. Previous works have addressed geometric modeling and calibration techniques for fisheye lenses to improve reconstruction reliability [8, 9].

Feature detection and matching are crucial steps in the SfM pipeline, as they establish correspondences between images for estimating camera poses and reconstructing 3D structures. In our study, we employed the Scale-Invariant Feature Transform (SIFT) algorithm for keypoint extraction and matching. However, fisheye images introduce significant radial distortions, affecting the accuracy and stability of feature correspondences. To mitigate this issue, we applied a distortion correction step before

feature extraction.

To evaluate the effectiveness of distortion correction, we conducted an experiment comparing feature matching results before and after undistortion. The comparison is shown in Figure 4.1, where green lines represent successfully matched keypoints.



Figure 4.1: Feature matching before (left) and after (right) undistortion. Green lines indicate successfully matched keypoints.

Before undistortion, radial distortions caused noticeable displacement of keypoints, leading to inconsistent feature matches, especially in peripheral image regions. This instability negatively impacted later stages such as camera pose estimation and 3D reconstruction.

After undistortion, the spatial distribution of keypoints became more uniform, reducing the disparity between matched features. The improved keypoint alignment provided a stronger foundation for subsequent steps. These findings align with previous studies emphasizing the importance of distortion correction in feature-based SfM pipelines [10].

4.2 3D Reconstruction Quality

Our results align with previous studies that explored fisheye-based 3D reconstruction techniques and their effectiveness in wide-FOV applications [11]. To assess reconstruction quality, we analyze the input scene and its reconstructed results. Figure 4.2 presents a sample frame from the video sequence, depicting an indoor environment with structured elements such as walls, doors, and posters. These structured elements provide strong visual features that facilitate keypoint detection and matching.



Figure 4.2: Input frame of the indoor scene.

The corresponding sparse reconstruction is illustrated in Figure 4.3. The reconstruction captures key geometric structures of the scene, but some areas, particularly those with uniform textures such as plain walls, exhibit a lower point density due to the reliance on feature-based methods. The colors in the sparse point cloud do not necessarily represent depth but are assigned randomly during visualization.

While the sparse reconstruction effectively preserves the spatial layout, it lacks fine details, leading to gaps in certain regions. To generate a more complete model, we perform dense reconstruction, as shown in Figure 4.4. This step refines the structural details and improves spatial consistency.

The dense reconstruction significantly enhances the completeness of the model, ensuring that previously fragmented structures are better connected. Fine details such as door frames and poster edges are more accurately reconstructed. However, regions with minimal texture still exhibit some level of reconstruction noise, a common limitation of feature-based methods.

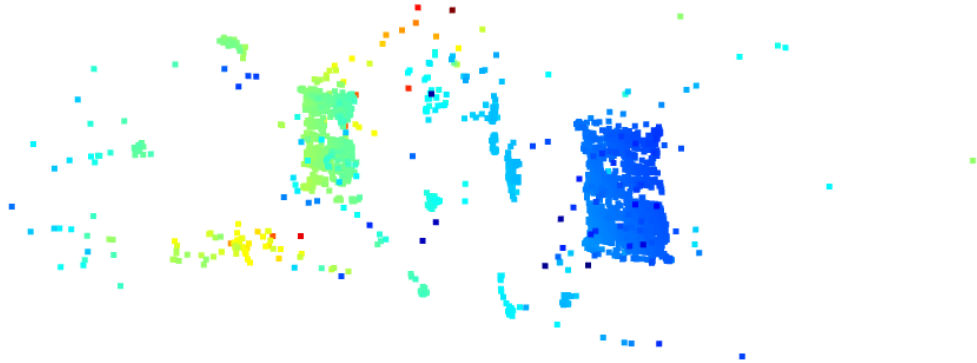


Figure 4.3: Sparse reconstruction of the indoor scene. Colors are randomly assigned for visualization.

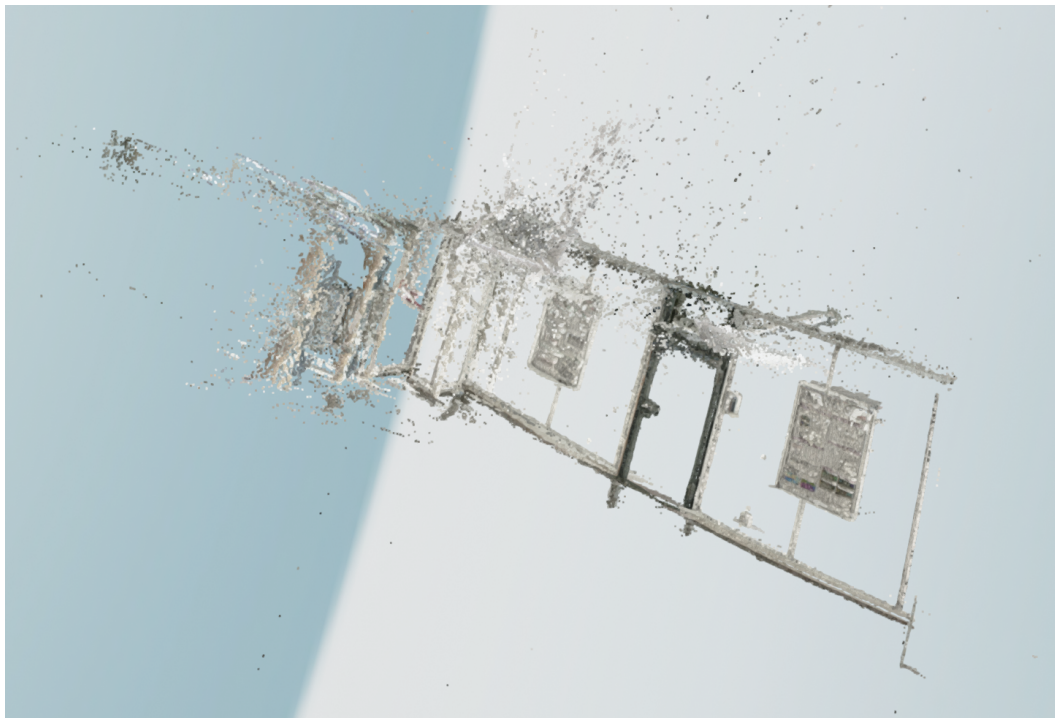


Figure 4.4: Dense reconstruction of the indoor scene.

4.2.1 Computational Efficiency

Computational efficiency is an important factor in large-scale 3D reconstruction tasks. Each stage of the Structure-from-Motion (SfM) pipeline has different computational demands, with dense reconstruction being the most time-consuming step. Understanding these computational costs helps identify bottlenecks and potential optimization strategies.

In our experiments, we processed a 600-frame video sequence and selected one image every 10 frames, resulting in a dataset of 60 images. Table 4.1 presents the processing time for different stages of the pipeline. While most steps remain computationally feasible, dense reconstruction requires significantly more time and computational resources.

Processing Step	Time (Seconds)
Calibration	35
Undistortion	20
Feature Matching	120
Sparse Reconstruction	180
Dense Reconstruction	3000

Table 4.1: Processing time breakdown for different pipeline stages, based on a dataset of 60 images.

Our experiments showed that computational time is highly dependent on hardware. On a lower-performance GPU, dense reconstruction required nearly 17 hours, whereas a high-performance GPU completed the same process in 2 to 3 hours. This highlights the importance of hardware selection and optimization strategies such as GPU acceleration and efficient parallel processing to improve scalability.

4.3 Impact of Scene Structure on Reconstruction Performance

Global SfM techniques have been developed to enhance the robustness of large-scale 3D reconstruction, particularly in environments with complex scene structures and varying lighting conditions [12]. To evaluate the robustness of our pipeline, we compare indoor and outdoor reconstructions.

To provide a better understanding of the original scene before reconstruction, Figure 4.5 shows three frames extracted from the input video. The frames are sampled at 10-frame intervals and stitched together to present a broader view of the environment captured by the fisheye camera. The scene consists of an open parking lot with

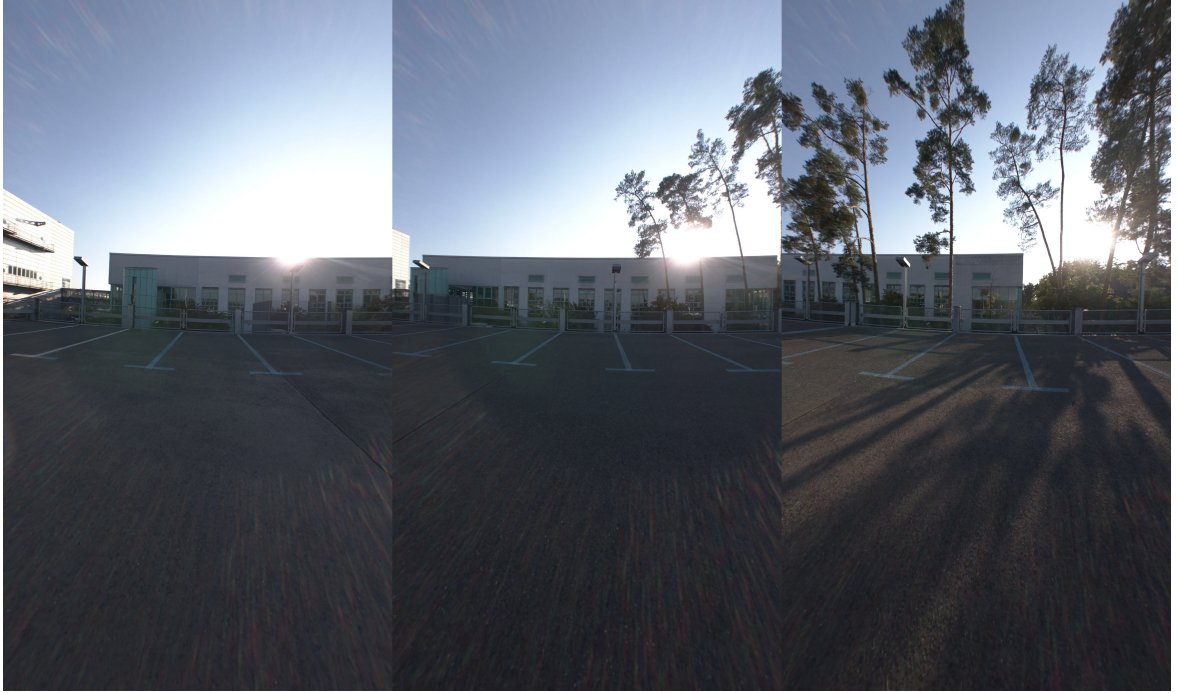


Figure 4.5: Extracted frames from the input video.

structured elements such as buildings, fences, and trees. These elements introduce different levels of texture and geometry, making it a suitable test case for evaluating the performance of our reconstruction pipeline.

One of the key challenges in outdoor reconstruction is handling large-scale scenes with varying depths. Unlike controlled indoor environments, outdoor settings often contain objects at significantly different distances from the camera, which can affect the accuracy of feature matching and depth estimation. Additionally, changes in lighting conditions, such as direct sunlight and shadows, introduce further difficulties in maintaining consistency in feature detection.

The dense reconstruction of the scene, shown in Figure 4.6, demonstrates how our pipeline processes such challenging environments. The reconstructed point cloud successfully captures major structural elements, such as the buildings and fences, while more complex, less textured areas, such as the sky and ground, exhibit lower point density. This behavior is expected in feature-based SfM pipelines, as regions with minimal texture often lack sufficient keypoints for robust triangulation.

Indoor scenes generally provide a denser set of feature points due to well-defined structures such as walls and posters, which offer strong geometric consistency. In contrast, outdoor scenes, especially those with vegetation, exhibit sparser feature dis-

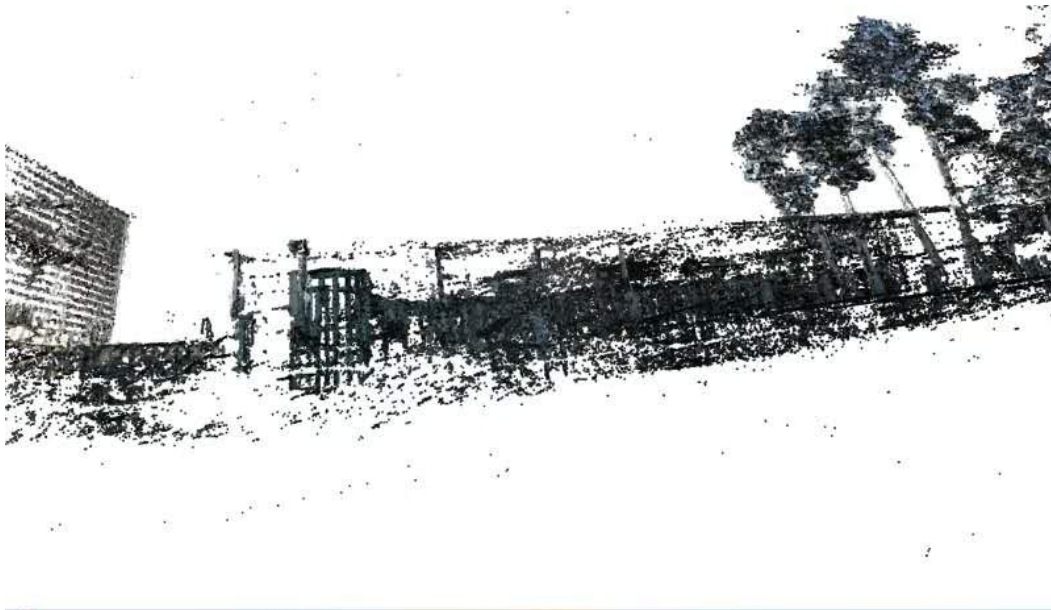


Figure 4.6: Dense reconstruction of the outdoor scene.

tributions due to the irregular nature of tree branches and other organic elements.

Lighting conditions also play a crucial role in reconstruction quality. Indoor environments typically have stable lighting, ensuring consistent feature detection across frames. However, outdoor lighting varies depending on the time of day, weather conditions, and the presence of dynamic shadows, all of which can introduce inconsistencies in keypoint matching.

Furthermore, the complexity of scene geometry significantly impacts reconstruction accuracy. Indoor environments often feature structured layouts with parallel lines and well-defined edges, which aid in establishing geometric constraints. On the other hand, outdoor environments include diverse and irregular shapes, such as trees and fences, making it more challenging to achieve spatial consistency in the reconstructed model.

These results confirm that our pipeline is adaptable to different environmental conditions and can generalize well across both structured indoor scenes and unstructured outdoor environments. The ability to process large-scale, complex scenes with varying illumination conditions demonstrates its robustness for practical applications in fields such as robotics, autonomous navigation, and 3D mapping.

Chapter 5

Conclusion

This work presents a fisheye-adapted Structure-from-Motion (SfM) pipeline designed to improve 3D reconstruction from fisheye video input. The proposed system integrates undistortion, optimized feature matching, and adapted sparse and dense reconstruction techniques. Through experimental evaluation, we successfully applied this pipeline to both indoor and outdoor environments, demonstrating its robustness in handling fisheye distortions and varying scene complexities.

A key outcome of this project is the successful integration of undistortion as a preprocessing step, which significantly enhances the spatial distribution of feature correspondences. Our results show that undistorted images yield more stable feature matching, particularly in peripheral regions where standard fisheye SfM pipelines struggle. This improvement directly enhances camera pose estimation and contributes to more consistent 3D reconstructions.

Additionally, we evaluated the pipeline in diverse environments, comparing structured indoor scenes with unstructured outdoor settings. Indoor reconstructions exhibit a higher density of feature correspondences due to well-defined geometric structures, while outdoor reconstructions face additional challenges, such as lighting variations and irregular shapes. Despite these challenges, our pipeline successfully reconstructs key structures in both scenarios, confirming its adaptability to different real-world applications.

Another major contribution of this work is the analysis of computational performance. We examined the impact of different hardware configurations on processing time, finding that GPU selection significantly affects reconstruction speed. Our experiments show that computation time varies from several hours to nearly a full day depending on the GPU used. This insight highlights the importance of hardware optimization for improving scalability, particularly in large-scale 3D reconstruction tasks.

Overall, this project successfully developed and validated an SfM pipeline specifically adapted for fisheye imagery. The system improves feature matching, enhances pose estimation, and enables structured 3D reconstruction across different environments.

The robustness of our approach demonstrates its potential for applications in robotics, augmented reality, and large-scale mapping.

List of Figures

2.1	Workflow of fisheye image processing and calibration.	2
2.2	Schematic diagram of fisheye lens distortion.	3
2.3	Structure-from-Motion (SfM) workflow.	7
2.4	Feature extraction and descriptor sampling region in SIFT.	8
2.5	Gradient computation and SIFT keypoint descriptor representation. . .	10
3.1	Extracted video frame from fisheye video input.	15
3.2	Fisheye calibration image using checkerboard pattern.	16
3.3	Chessboard calibration visualization.	17
3.4	Distortion correction result	18
3.5	Extracted keypoints using SIFT	19
3.6	Feature matching results after RANSAC	20
4.1	Feature matching before (left) and after (right) undistortion. Green lines indicate successfully matched keypoints.	25
4.2	Input frame of the indoor scene.	26
4.3	Sparse reconstruction of the indoor scene. Colors are randomly assigned for visualization.	27
4.4	Dense reconstruction of the indoor scene.	27
4.5	Extracted frames from the input video.	29
4.6	Dense reconstruction of the outdoor scene.	30

List of Tables

4.1 Processing time breakdown for different pipeline stages, based on a
dataset of 60 images. 28

Bibliography

- [1] J. Kannala and S. Brandt, “A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 28, no. 8, pp. 1335–1340, 2006.
- [2] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [3] Kaup.A., *Scale space representation in Image, Video, and Multidimensional Signal Processing*. Internal educational material, 2024.
- [4] P. Torr and A. Zisserman, “Mlesac: A new robust estimator with application to estimating image geometry,” *Computer Vision and Image Understanding*, vol. 78, no. 1, pp. 138–156, 2000.
- [5] X. Gao, T. Zhang, Y. Liu, and Q. Yan, “14 lectures on visual slam: from theory to practice,” *Publishing House of Electronics Industry*, pp. 206–234, 2017.
- [6] J. L. Schönberger and J.-M. Frahm, “Structure-from-motion revisited,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4104–4113.
- [7] T. Tuytelaars, K. Mikolajczyk *et al.*, “Local invariant feature detectors: a survey,” *Foundations and trends in computer graphics and vision*, vol. 3, no. 3, pp. 177–280, 2008.
- [8] E. Schwalbe, “Geometric modelling and calibration of fisheye lens camera systems,” in *Proc. 2nd Panoramic Photogrammetry Workshop, Int. Archives of Photogrammetry and Remote Sensing*, vol. 36, no. Part 5. Citeseer, 2005, p. W8.
- [9] M. Kedzierski and A. Fryskowska, “Precise method of fisheye lens calibration,” in *Proceedings of the ISPRS-Congress, Beijing, China*, 2008, pp. 765–768.

- [10] J. L. Schonberger and J.-M. Frahm, “Structure-from-motion revisited,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 4104–4113.
- [11] C. Ma, L. Shi, H. Huang, and M. Yan, “3d reconstruction from full-view fisheye camera,” *arXiv preprint arXiv:1506.06273*, 2015.
- [12] P. Moulon, P. Monasse, and R. Marlet, “Global fusion of relative motions for robust, accurate and scalable structure from motion,” in *Proceedings of the IEEE international conference on computer vision*, 2013, pp. 3248–3255.